GitHub Flow : le bon workflow

Branche secondaire, Pull Request, contrôles automatiques et merge sur le main protégé

Aide-mémoire pour le lab gh200-azure-lab | Préparation GH-200

0 UNE SEULE FOIS : protège le main

Settings > Rules > Rulesets, cible main. Exige la PR, le status check "test" au vert et au moins 1 approbation (0 si tu travailles seul). Bloque les force push.

```
Settings > Rules > Rulesets
[x] Require a pull request
[x] Require status checks: test
[x] Block force pushes
```

1 Crée la branche de travail

Ne travaille jamais directement sur main : pars d'une branche secondaire.

```
git switch main
git pull origin main
git switch -c dev
```

2 Travaille, teste en local, commit

Les tests locaux détectent les erreurs tout de suite, mais ne remplacent pas la CI.

```
pytest
git add .
git commit -m "feat: ..."
```

3 Publie la branche sur GitHub

Le -u relie dev en local à origin/dev : ensuite un simple git push suffit.

```
git push -u origin dev
```

4 Ouvre la Pull Request dev vers main

Sur GitHub : Compare & pull request. Dès l'ouverture, la CI (on: pull_request) se lance sur le code de la PR.

```
GitHub > Pull requests
> New pull request
base: main  <-  compare: dev
```

5 CI verte + approbation

Le bouton Merge ne s'active que si les checks passent et que la review est arrivée (CODEOWNERS si présents). Si la CI échoue : corrige, commit et push, la PR se met à jour toute seule.

```
git add . && git commit
git push        # met à jour la PR
```

6 Merge sur le site et réalignement du local

Fais le merge sur GitHub (Merge / Squash / Rebase), puis récupère main à jour et nettoie.

```
git switch main
git pull origin main
git branch -d dev
```

Après le merge : le CD part depuis main

déploiement staging > approbation manuelle (environment production) > déploiement production, avec login OIDC vers Azure.

Pourquoi la PR et pas le merge en local

- La CI tourne sur un runner propre : elle trouve les erreurs du type "chez moi ça marche".
- Les tests partent AVANT le merge : le main reste toujours propre.
- Review et traçabilité : on garde la trace de qui a approuvé et pourquoi.
- Le CD part depuis main : seul du code vérifié arrive en production.

Erreurs fréquentes

- Merge en local puis push sur un main protégé : erreur GH006. Solution : git reset --hard origin/main, puis passe par la PR.
- Le status check n'apparaît pas dans les checks requis : le workflow doit avoir tourné au moins une fois. Avec la matrix, le nom est test (3.11), etc.
- Seul, tu ne peux pas approuver ta propre PR : mets 0 approbation dans le ruleset.

Git = merge, branches, historique (local) | GitHub = Pull Request, review, status check, ruleset (distant)